

TASK2 - RGB CHANNEL VISUALIZATION WITH HISTOGRAMS

СУТЬ ЗАДАЧИ

Task2 реализует разделение изображения на три цветовых канала (красный, зеленый, синий) и отображение каждого канала рядом с соответствующей гистограммой распределения яркости.

Основная цель: Позволить пользователю анализировать распределение цветовых значений в каждом канале RGB-изображения через визуализацию каналов и их гистограмм.

АРХИТЕКТУРА TASK2

Структура класса (tasks.h):

```
class Task2 : public TaskInterface {
public:
    // Основные данные
    ImageRGB imgR, imgG, imgB;           // Выделенные каналы

    // Текстуры для GPU-рендеринга
    SDL_Texture* texR = nullptr;
    SDL_Texture* texG = nullptr;
    SDL_Texture* texB = nullptr;

    // Гистограммы для каждого канала
    std::array<int, 256> histR{}, histG{}, histB{};

    // Методы интерфейса
    void prepare(const AppContext& ctx) override;
    void drawByTexture(const AppContext& ctx) override;
    void drawByPixels(const AppContext& ctx) override;
};
```

Членские переменные:

- imgR, imgG, imgB (ImageRGB): RGB-изображения каждого канала (R в красном, G в зеленом, B в синем)
- texR, texG, texB (SDL_Texture*): GPU-текстуры для быстрого рендеринга через drawByTexture()
- histR, histG, histB (std::array<int, 256>): Гистограммы яркости каждого канала (индекс = значение 0-255, значение = количество пикселей)

ЖИЗНЕННЫЙ ЦИКЛ TASK2

1. prepare() – Инициализация (вызывается один раз при открытии Task2)

Шаг 1: Проверка входных данных (renderer, image)

Шаг 2: Выделение каналов R, G, B из исходного изображения

```
imgR = extractChannelR(image);  
imgG = extractChannelG(image);  
imgB = extractChannelB(image);
```

Шаг 3: Вычисление гистограмм для каждого канала

```
histR = histogramChannelR(image);  
histG = histogramChannelG(image);  
histB = histogramChannelB(image);
```

Шаг 4: Создание SDL_Texture для GPU-рендеринга

```
texR = makeTextureRGB(ctx.renderer, imgR);  
texG = makeTextureRGB(ctx.renderer, imgG);  
texB = makeTextureRGB(ctx.renderer, imgB);
```

2. drawByTexture() – Быстрый рендер через текстуры (вызывается каждый кадр)

Вызывается каждый кадр для отрисовки результата через GPU-текстуры (быстро).

Алгоритм:

- Расчет макета: $blockW = (avail.x - gap) * 0.5f$ (ширина для изображения)
 $blockH = (avail.y - ...) / 3.0f$ (высота для одного канала)
- Для каждого канала (R, G, B):
 - Отображает выделенный канал слева (ImGui::Image)
 - Отображает гистограмму справа (drawHistogramByTexture)

3. drawByPixels() – Ручной рендер по пикселям (альтернатива drawByTexture)

Вызывается в режиме ручного рендеринга.

Алгоритм:

- Установка области отрисовки (clip rect)
- Расчет размеров:
 - $blockW = (areaW - gap) * 0.5f$ (половина для изображения, половина для гистограммы)
 - $blockH = (areaH - ...) / 3$ (для трех строк R, G, B)
- Для каждого канала:
 - Отрисовка изображения канала: Двойной цикл по пикселям, масштабирование и рисование через SDL_RenderPoint()
 - Отрисовка гистограммы:
 1. Находит максимальное значение в гистограмме

2. Масштабирует высоту гистограммы по blockH
 3. Для каждого из 256 значений яркости рисует столбец нужного цвета (красный/зеленый/синий)
 4. Столбец рисуется от низа вверх высотой, пропорциональной количеству пикселей
- Отключение clip rect

ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ (image_processing.cpp)

extractChannelR/G/B()

Назначение: Выделить один цветовой канал из RGB-изображения в отдельное изображение, где канал отображается в его собственном цвете.

Пример (extractChannelR):

```
// Входное изображение: RGB пиксели [R, G, B]
// Выходное изображение: [R, 0, 0] (только красная компонента)

ImageRGB extractChannelR(const ImageRGB& img) {
    ImageRGB result(img.width, img.height);
    for (int y = 0; y < img.height; ++y) {
        for (int x = 0; x < img.width; ++x) {
            uint8_t r = img.at(x, y)[0];
            result.at(x, y)[0] = r;    // R компонента
            result.at(x, y)[1] = 0;    // G = 0
            result.at(x, y)[2] = 0;    // B = 0
        }
    }
    return result;
}
```

histogramChannelR/G/B()

Назначение: Подсчитать распределение значений яркости в одном канале.

Как работает:

1. Создает массив из 256 нулей (по одному для каждого значения 0-255)
2. Проходит по всем пикселям изображения
3. Для каждого пикселя берет значение канала (0-255)
4. Увеличивает счетчик в позиции массива для этого значения

Пример:

```
// Если в изображении 1000 пикселей с красным значением 100
// To hist[100] = 1000
// Это означает: "1000 пикселей имеют красную интенсивность 100"
```

drawHistogramByTexture()

Назначение: Нарисовать гистограмму через ImGui с использованием низкоуровневых команд отрисовки.

Параметры:

- dl – DrawList (список команд отрисовки ImGui)
- origin – верхний левый угол гистограммы (экранные координаты)
- size – размер области гистограммы (ширина и высота)
- hgt – массив значений гистограммы (256 элементов)
- color – цвет столбцов гистограммы (IM_COL32)

ЦВЕТОВЫЕ КАНАЛЫ В ГИСТОГРАММАХ

Red (R): Красный (255, 0, 0) - Показывает распределение красной компоненты в пикселях

Green (G): Зеленый (0, 255, 0) - Показывает распределение зеленой компоненты в пикселях

Blue (B): Синий (0, 0, 255) - Показывает распределение синей компоненты в пикселях

ПРИМЕР: ИНТЕРПРЕТАЦИЯ ГИСТОГРАММ

Сценарий: Яркое красное изображение

Исходное изображение: RGB пиксели типа (255, 100, 100)

Выделенные каналы:

- Red: (255, 0, 0) – Ярко-красное изображение
- Green: (100, 100, 100) – Темно-серое изображение
- Blue: (100, 100, 100) – Темно-серое изображение

Гистограммы:

- Красная гистограмма: Один высокий столбец в позиции 255 (все пиксели красные)
- Зеленая гистограмма: Один столбец в позиции 100 (все пиксели имеют зеленую = 100)
- Синяя гистограмма: Один столбец в позиции 100 (все пиксели имеют синюю = 100)

ИЗМЕНЕНИЯ В DRAWBYPIXELS (ОБНОВЛЕНИЕ)

Проблема: drawByPixels() отображал только изображения каналов без гистограмм

Что было изменено:

1. Разделение ширины блока

ДО:

```
float blockW = areaW; // Все пространство занимает изображение
```

ПОСЛЕ:

```
const float gap = 8.0f;
```

```
float blockW = (areaW - gap) * 0.5f; // Половина для изображения, половина для гистограммы
```

2. Добавлены данные для гистограмм

```
// Массивы указателей на гистограммы
std::array<int, 256>* hists[3] = { &histR, &histG, &histB };

// Цвета для каждой гистограммы
Uint8 histColors[3][3] = {
{ 255, 0, 0 }, // Red
{ 0, 255, 0 }, // Green
{ 0, 0, 255 } // Blue
};
```

3. Добавлена отрисовка гистограмм

```
// После отрисовки каждого канального изображения...
```

```
// Отрисовка гистограммы справа
const std::array<int, 256>& hgt = *hists[k];
```

```
// Найти максимум для масштабирования
int maxH = 0;
for (int v : hgt) if (v > maxH) maxH = v;
if (maxH <= 0) continue;
```

```
// Масштабирование и позиционирование
float scale = blockH / (float)maxH;
float hx = originX + blockW + gap; // Начало гистограммы справа
float dxh = blockW / 256.0f; // Ширина столбца
```

```
// Установка цвета (красный, зеленый или синий)
Uint8 cr = histColors[k][0];
Uint8 cg = histColors[k][1];
Uint8 cb = histColors[k][2];
SDL_SetRenderDrawColor(renderer, cr, cg, cb, 255);
```

```
// Рисование 256 столбцов гистограммы
for (int v = 0; v < 256; ++v) {
float xv = hx + (v + 0.5f) dxh;
float barH = hgt[v] scale;
if (barH < 1.0f && hgt[v] > 0) barH = 1.0f;
```

```
    for (int py = 0; py < (int)barH; ++py)
        SDL_RenderPoint(renderer, xv, blockTop + blockH - py);
```

```
}
```

Результат:

Теперь drawByPixels() отображает:

- Слева: Выделенные RGB каналы (красное, зеленое, синее изображения)

- Справа: Цветные гистограммы каждого канала
- Макет идентичен `drawByTexture()`
- Оба режима работают корректно